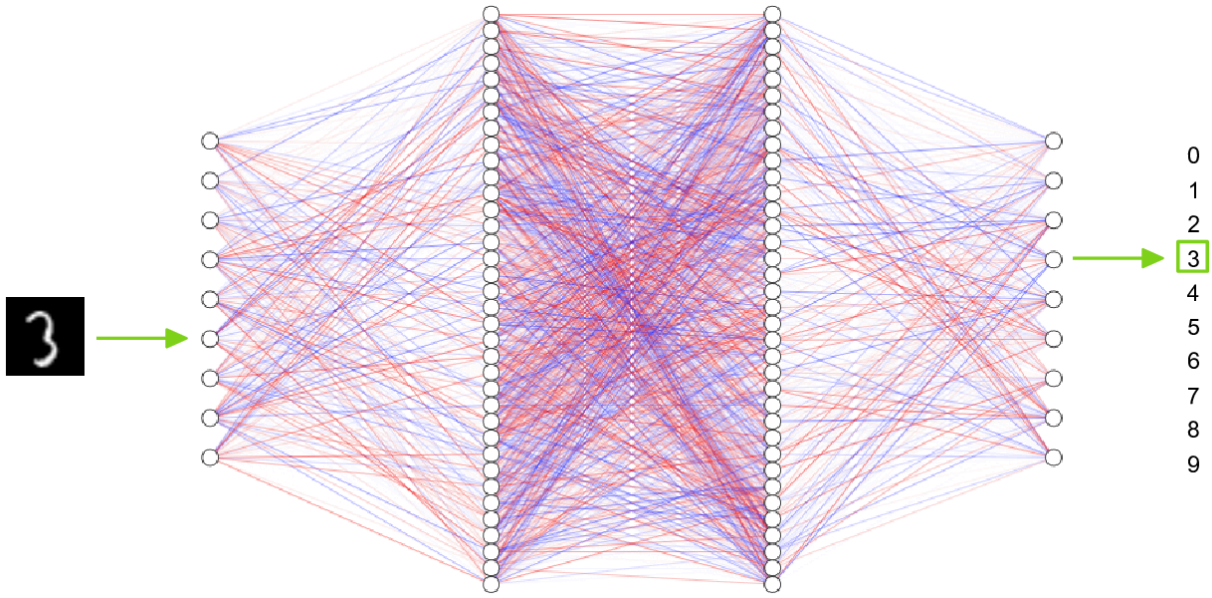
Full link to blog :- [akisa.vujel]

Introduction to neural network



A neural machine is a machine learning model that mimics the functionality of a human brain. These models are interconnections of nodes or neurons that recognize patterns, process data and decision making. Neural network is a key aspect of artificial intelligence that gives machine learning algorithms to make accurate predictions. These models are built on some components :-

1. Neurons :- Main unit of machine which takes input.
2. Connections :- Links between neurons that carry information.
3. Weight and Biases :- Parameters to determine strength and influence on connection.
4. Propagation function :- Mechanism to transfer data through layers of neurons.
5. Learning rule :- Method to adjust weight and biases to improve accuracy.

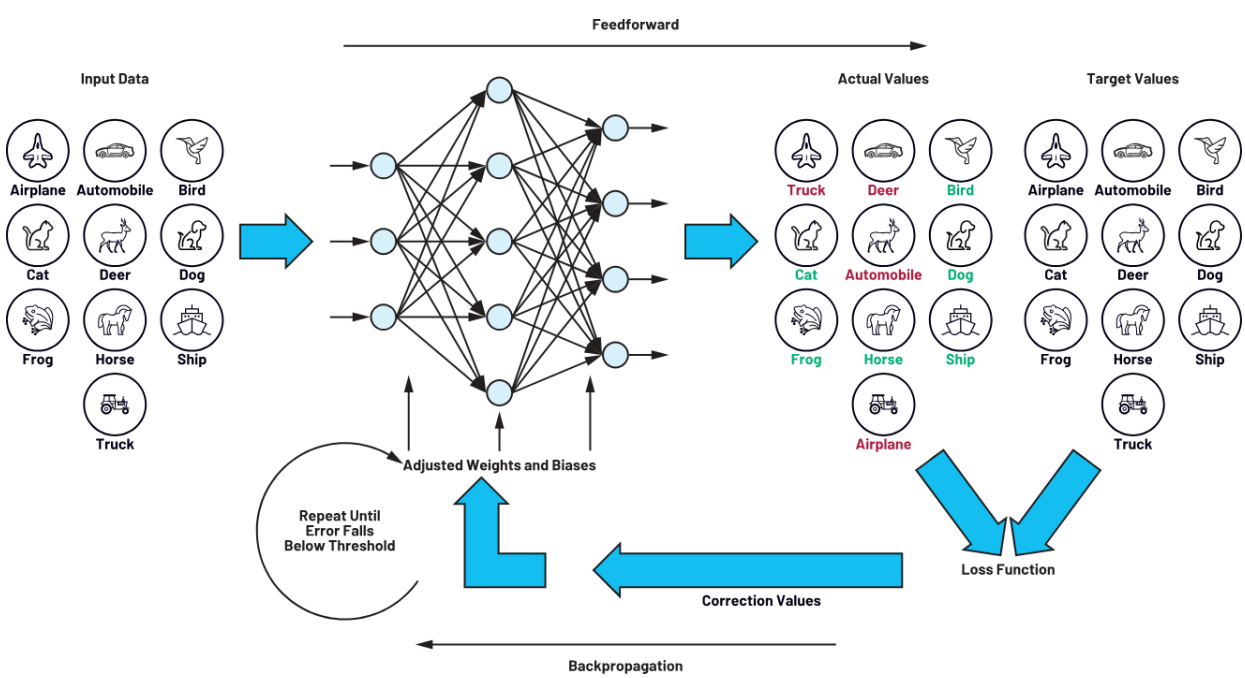
Neurons receive multiple input signals that add up and process sum to reach a threshold using sigmoid function, if the function produces a value that works then it becomes the output. Network works with multiple layers to create a network.

1. Input layer :- Takes input data and translates it to hidden layer of neuron using synapsis, input layers have many nodes as input columns of data.
2. Hidden layer :- Categories the data received from input layer, nodes in hidden layer sends data to more hidden layers then finally, in output layer. The hidden layer in an artificial neural network is called a “black box” because researchers cannot determine the result.
3. Output layer :- output layer has multiple nodes, takes data from hidden layers and gives results.

How neural network is trained

Neural network trains with the networking processing large amounts of data with outputs. Neural network is supervised learning with labeled data known as output then produces correct results for unknown data.

Weight is the strength of connection in nodes, during training the network is assigned high or low weight. Strengthen the signal as the weight in nodes increases, the weight adjusted using gradient descent method that calculates difference between actual value and predicted value. The error gets minor as the weight increases.



Core concepts of neural network

1. **Forward propagation**
   The data inside the neural network modes in forward direction from input layer to output layer this process is known as forward propagation.

2. **Linear transformation**
   Each neuron is assigned with an input that is multiplied by weight associated with connection. These product summed together and biases is represented with

   $Z = w_1x_1 + w_2x_2 + \ldots + w_{\square}x_{\square} + b$
   Where,
   w represent weight
   x represent inputs
   b represent biases

3. **Activation**
   The linear transformation also known as Z is passed through an activation function. Activation enables the network to work with complex patterns. Popular activation functions like sigmoid, ReLU and tanh

4. **Backpropagation**
   The network evaluates its performance using a loss function which measures between actual output and predicted output. The goal is to minimize the loss; this is where backpropagation comes.
   1. Loss calculation :- Calculate the loss error in predicted value, common choice of function can be mean squared error.
   2. Gradient calculation :- The network computes the gradient of loss function to each weight and biases in the network, this involves chain rule of calculus to find which part of error can be attributed to each weight and biases.
   3. Weight update :- After gradient calculation biases and weight are updated using stochastic gradient descent (SGD).

5. **Iteration**
   The process of forward propagation, linear transformation, activation, back propagation is repeated many times. This process reduced the error and gave a more accurate prediction. Through these steps neural network parameters adapt to better relationships in data.

Implementation of neural network

MNIST

Neural Network

✦ HANDWRITTEN DIGIT RECOGNITION ✦

Step 1 :- Import necessary libraries and dataset

```
import numpy as np
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
```

Step 2 :- Normalize and Flatten the Image Data

```
X_train = X_train/255
X_test = X_test/255

X_Train_flat = X_train.reshape(len(X_train),28*28)
X_Test_flat = X_test.reshape(len(X_test),28*28
```

Step 3 :- Implementing a Neural Network with Keras

```
model = keras.Sequential([
   keras.layers.Dense(10, input_shape=(784,), activation='sigmoid')
])

model.compile(optimizer='adam',
          loss='sparse_categorical_crossentropy',
          metrics=['accuracy'])

model.fit(X_Train_flat, y_train, epochs=5)
```

Step 4 :- Evaluating and Making Predictions with the Neural Network

```
model.evaluate(X_Test_flat, y_test)
y_pred = model.predict(X_Test_flat)
```

Step 5 :- Understanding Model Performance

```
np.argmax(y_pred)
np.int64(90600)
y_predicted_labels = [np.argmax(i) for i in y_pred]

Print(y_test[:7])
Print(y_predicted_labels)
```

Step 6 :- Plotting the Confusion Matrix

```
cm = tf.math.confusion_matrix(labels=y_test,predictions=y_predicted_labels)

plt.figure(figsize = (10,7))
sns.heatmap(cm,annot = True, fmt = 'd')
plt.xlabel('predicted')
plt.ylabel('Truth')
```

THANK YOU